

∴ {cover}

RHEL 9 · RHCSA 实操讲义

06

第 06 章 复制、归档、压缩与远程传输

从源集合到目标集合：把层级、成员、属性、远端路径和完整性证据放进同一条操作链。

对象模型操作语义验证诊断经典任务

大字号阅读版

⋮

⋮ {reading-nav}

本章阅读导航

先抓住一条主线：文件操作不是“把一个名字搬过去”，而是把源对象或成员集合，按照明确的层级、属性和删除边界，变换为可验收的目标集合。

01 确认源集合对象、成员、隐藏项、链接

02 定义目标层级目录本身还是目录内容

03 选择变换方式复制、部署、归档、传输、同步

04 控制风险边界覆盖、属性、删除、远端路径

05 执行最小操作先基线或 dry-run，再改变状态

06 分层验收成员、路径、字节、元数据

专题地图

- 知识专题 数据集合、路径层级与元数据
- 操作专题 ``cp` / `mv`` 与覆盖边界
- 操作专题 ``install`` 部署文件
- 知识专题 ``tar`` 成员、路径与元数据
- 操作专题 压缩、查看和安全提取
- 操作专题 ``scp` / `sftp`` 远端路径
- 操作专题 ``rsync`` 增量同步
- 操作专题 ``sha256sum`` 与分层验收
- 诊断专题 从症状推进到下一条证据
- 经典任务 归档试提取与远端同步

阅读时持续回答

1. 源是一个对象，还是一个成员集合？
2. 目标存在时，层级会怎样改变？
3. 归档、压缩、传输、同步分别改变哪一层？
4. 远端相对路径由谁解释？
5. ``rsync`` 源末尾斜杠改变了什么？
6. 哪些属性必须保留，执行身份是否允许？
7. 命令成功后，还缺哪一层证据？

章节边界：SSH 密钥与 ``ssh`` 留给第 20 章；NFS 留给第 26 章；本章不把 ``rsync`` 扩展为完整备份体系。

...

第 06 章 • 正文

系统管理员面对的并不是孤立的“文件”，而是一个由路径名称、目录层级、普通文件字节、链接和元数据共同组成的数据集合。把这个集合复制到同一台主机的另一个位置、封装为归档、压缩后交付，或者同步到远端，看似都叫“搬文件”，实际改变的对象、风险和验收方式并不相同。

最常见的误判并不是命令拼错，而是对象边界理解错了：复制了目录本身而不是目录内容；归档里多出或少了一层目录；远端相对路径被解释到意外位置；`rsync` 因一个末尾斜杠改变了同步根；摘要一致却误以为权限、所有者和时间戳也一定一致；命令退出为零，却没有检查目标端实际状态。

本章以“源集合 → 变换或传输 → 目标集合 → 分层证据”为主线。前一章已经建立查找和筛选源对象的方法，本章从已经确定的源集合开始；下一章将进入用户、组和账号生命周期，本章只把现有执行身份作为属性和远端路径的一项约束。

⋮ {concept-box} 概念 源对象与目标对象 是一次文件操作的两端。源对象回答“读取什么”，目标对象回答“最终应出现在哪里以及以什么层级出现”。目标是否已存在、目标是文件还是目录，会改变同一条命令的结果，所以在覆盖、移动或同步前，应先用 `stat`、`test` 或目录树证据确认两端，而不是只凭命令外形推测。⋮

⋮ {concept-box} 概念 归档成员（archive member）是 `tar` 归档内部可列出、可提取的名称和对象。创建归档时，本地路径会被转换为成员名称；提取时，成员名称决定恢复出的顶层和目录结构。归档文件能够打开，只证明容器可读，不证明成员前缀、隐藏项和恢复位置符合任务要求。⋮

⋮ {concept-box} 概念 压缩流（compressed stream）是对一个文件或字节流进行编码后的表示。`gzip`、`bzip2` 和 `xz` 可以压缩 `tar` 产生的归档流，但它们本身不负责把多个目录项组织成成员集合。因而“归档”和“压缩”必须分开理解：先决定成员和层级，再决定是否以及如何压缩。⋮

⋮ {concept-box} 概念 远端路径（remote path）由远端登录身份和远端文件系统解释。`host:relative/path` 不是本地当前目录的延伸；它通常从远端用户的起始目录解释。可靠做法是显式写出用户和绝对目标路径，或在 `sftp` 中先用 `pwd` 与 `lpwd` 分清两端。⋮

⋮ {concept-box} 概念 增量同步（incremental synchronization）是比较源集合与目标集合后，只传输或调整被判定为不同的对象。`rsync` 的源末尾斜杠决定同步根，`--delete` 决定是否清理目标额外成员；它们改变的不是显示格式，而是目标树本身，因此必须在执行前通过 `dry-run` 审查。⋮

⋮ {concept-box} 概念 完整性校验（integrity verification）用摘要判断被比较字节是否一致。`sha256sum` 相同可以证明指定文件的字节相同，却不能证明路径、类型、模式、所有者、时间戳、ACL 或安全上下文相同。一个完整验收必须把成员、层级、字节和元数据证据组合起来。⋮

⋮ {semantic-map}

操作语义速查

先建立关键接口地图，再进入正文细节。以下命令组覆盖本章最常用的查询、修改和验证入口；参数采用纵向定义，避免把语义压成一长行。

`cp` / `mv`

SYNOPSIS

```
cp [OPTION]... SOURCE DEST
cp [OPTION]... SOURCE... DIRECTORY
mv [OPTION]... SOURCE DEST
mv [OPTION]... SOURCE... DIRECTORY
```

`cp` 创建副本并保留源；`mv` 改变名称或位置。两者都受目标是否存在、目标类型和覆盖策略影响。

重要参数 / 形式

`cp -r` / `cp -R`

递归复制目录树。

`cp -p`

请求保留模式、所有权和时间戳；实际结果仍受权限和文件系统限制。

`cp -a`

归档式递归复制，适合保留链接和常用属性；不等于已证明所有属性落地。

`SOURCE/.`

明确复制已有源目录的全部成员，包括隐藏成员，进入既定目标目录。

`install`

SYNOPSIS

```
install [OPTION]... SOURCE DEST
install -d [OPTION]... DIRECTORY...
```

把准备好的文件部署到目标位置，并在同一操作中表达目标目录、模式、所有者和组。

重要参数 / 形式

`-D`

创建目标文件缺失的父目录。

`-d`

创建目录对象，而不是复制普通文件。

-m MODE

显式设置目标模式。

-o OWNER / -g GROUP

设置目标所有者或组，需要相应权限。

tar

SYNOPSIS

```
tar -cf ARCHIVE MEMBERS...
tar -tf ARCHIVE
tar -xf ARCHIVE [-C DIRECTORY]
```

把多个成员及其元数据组织为归档，或非破坏性查看成员，再提取到受控目录。

重要参数 / 形式

-c

创建归档。

-t

列出成员，不提取。

-x

提取成员。

-f ARCHIVE

指定归档文件；归档名紧随 **-f**。

-C DIRECTORY

改变后续路径参数的解释位置，具有顺序语义。

-z / -j / -J

分别选择 gzip、bzip2、xz 压缩。

gzip / bzip2 / xz

SYNOPSIS

```
gzip [OPTION]... FILE...
bzip2 [OPTION]... FILE...
xz [OPTION]... FILE...
```

压缩单个文件或字节流；目录树需要先通过 **tar** 形成成员集合。

重要参数 / 形式

`-d`

解压。

`-k`

保留输入文件，避免默认替换原文件。

`file ARCHIVE`

检查实际格式；后缀只是命名约定。

scp / sftp

SYNOPSIS

```
scp [OPTION]... SOURCE... TARGET
sftp [user@]host
```

通过既有 SSH 通道复制或交互传输。本章只处理客户端路径语义，不展开密钥和 `sshd` 配置。

重要参数 / 形式

`[user@]host:path`

远端操作数；相对路径由远端登录语境解释。

`scp -r`

递归复制目录。

`scp -p`

请求保留修改时间、访问时间和模式。

`pwd / lpwd`

在 `sftp` 中分别查看远端和本地工作目录。

rsync

SYNOPSIS

```
rsync [OPTION]... SOURCE... DEST
rsync [OPTION]... SOURCE... [user@]host:DEST
```

比较源集合与目标集合并同步差异。源末尾斜杠和删除选项直接改变目标树。

重要参数 / 形式

`-a`

归档模式，常用组合为递归并保留链接、模式、时间、所有者、组和设备信息；不包含 `-HAX`。

`-v`

输出较详细的处理信息。

`-n / --dry-run`

只预览，不改变目标。

`-i`

逐项显示变化，适合审查计划。

`--delete`

删除接收端的额外成员，只用于题目明确要求镜像时。

`SOURCE/` 与 `SOURCE`

前者同步目录内容；后者通常把源目录本身放入目标。

`sha256sum`

SYNOPSIS

```
sha256sum [OPTION]... FILE...
sha256sum -c CHECKSUM_FILE
```

为普通文件字节建立摘要证据，或按清单检查；不能代替路径、成员和元数据验收。

重要参数 / 形式

`sha256sum FILE`

计算文件摘要。

`sha256sum -c SHA256SUMS`

按清单检查文件字节。

`stat PATH`

与摘要配合，检查类型、模式、所有者和时间戳。

...

[知识专题] 数据集合、路径层级与元数据：复制的对象到底是什么

复制或传输前，最顺的切入点不是立刻选择 `cp` 还是 `rsync`，而是把目标改写成可观察的状态：源集合包含什么，目标应该出现什么，是否允许保留额外成员，需要保留哪些元数据，以及哪类证据才足以验收。只有先确定这些问题，目录末尾斜杠、归档成员前缀和远端路径才有明确含义。

① [知识点] 字节、名称与元数据是三个独立状态维度

普通文件最直观的是字节内容，但系统管理任务通常还关心：

- 路径名称和目录层级；
- 文件类型，是普通文件、目录还是符号链接；
- 访问模式、所有者和组；
- 修改时间等时间戳；
- ACL、扩展属性和 SELinux 上下文；

- 硬链接关系。

`sha256sum` 相同不代表这些维度相同。反过来，两个文件的大小和时间相同，也不能独立证明字节相同。验收时应根据题目明确要求选择证据，而不是把一个局部指标扩大成“完全一致”。

② [知识点] 目录本身与目录内容不是同一个复制目标

假设源为 `/srv/source`，其中有 `a.conf` 和隐藏成员 `.env`。常见意图有两种：

复制目录对象：目标中形成 `source/a.conf` 和 `source/.env`
复制目录内容：`a.conf` 和 `.env` 直接进入既定目标目录

`cp`、`tar` 和 `rsync` 对源操作数的解释方式不同，但共同原则是：不要用模糊通配符代替明确的集合边界。`/srv/source/*` 通常漏掉点号开头的成员；复制全部内容时，`/srv/source/.` 比 `*` 更能直接表达意图。

③ [知识点] 目标是否存在会改变结果层级

执行前至少回答：

目标不存在？
目标是普通文件？
目标是已有目录？
目标目录中是否已有同名成员？

例如把一个文件复制到不存在的路径会创建该路径；复制到已有目录会在目录中使用源基名。复制目录时，目标目录是否已经存在可能决定是创建目标目录，还是在目标目录内再形成一层源目录。稳定做法是先用 `stat` 或 `test` 建立目标基线，再根据预期树形结果选择命令。

④ [知识点] 命令成功只是执行状态，不是交付终态

退出状态为零通常说明命令按自身规则完成，但不能证明：

- 隐藏成员已包含；
- 归档顶层正确；
- 远端文件落在预期目录；
- 所有者和时间戳得到保留；
- 目标没有意外额外成员；
- `--delete` 没有超出授权范围。

因此每个操作必须预先写出验收证据，而不是执行后再凭感觉挑几条命令。

⑤ [知识点] “保留属性”表示操作意图，最终仍受身份和文件系统约束

`cp -a`、`scp -p`、`rsync -a` 或 `tar` 的保留选项都表达一种保留策略，但实际能否写入所有者、组、ACL、扩展属性或安全上下文，还取决于：

- 目标端执行身份；
- 目标文件系统是否支持该属性；
- 中间协议和工具是否携带该属性；
- 提取时的用户和 umask；
- 是否显式启用了 ACL、xattr 或 SELinux 元数据支持。

因此参数说明应写成“请求保留哪些属性”，验证说明应写成“目标端实际是什么”。

[Cheatsheet] 先拆分字节、层级和元数据；复制目录前确定是目录本身还是成员集合；目标存在状态会改变结果；退出状态只证明局部；保留参数不替代目标端 `stat`、成员检查和摘要检查。

[操作专题] 使用 `cp` 与 `mv` 安全复制、改名和移动

`cp` 和 `mv` 都接受源与目标，但它们改变的对象不同。`cp` 新建一个副本，源仍然存在；`mv` 改变名称或位置，成功后通常不再保留原路径。操作前应先确认源类型、目标类型、覆盖风险和需要保留的属性，操作后则分别检查两端。

① [操作] 使用 `cp` 复制单个文件

作用对象：一个普通文件及其目标路径。

基本形式：

```
cp SOURCE DESTINATION
cp SOURCE... DIRECTORY
```

单个源和不存在的目标路径会创建一个新文件；多个源必须以目录作为目标。复制前可用：

```
stat -- SOURCE DESTINATION 2>/dev/null
```

建立源和目标基线。若目标已有同名文件，默认行为通常是覆盖目标内容，因此高风险位置应先备份或使用交互确认，而不是把强制覆盖作为默认操作。

验证：

```
stat -- DESTINATION
sha256sum -- SOURCE DESTINATION
```

第一条确认对象和元数据，第二条只确认字节。

② [操作] 递归复制目录并选择属性策略

普通递归复制可使用 `cp -R` 或 `cp -r`。需要尽可能保留链接、模式、所有权、时间戳和扩展属性时，常用：

```
cp -a SOURCE DESTINATION
```

`-a` 是归档复制模式，适合已有目录树迁移，但并不意味着无需验证。普通用户无法把任意所有者写入目标；目标文件系统也可能不支持源端的全部属性。

③ [操作] 复制目录内容时使用 `SOURCE/.`

要把源目录的所有成员，包括隐藏成员，直接放入已有目标目录：

```
cp -a /srv/source/. /srv/destination/
```

这里的 `.` 是源目录自身的目录项语义，避免了 `/srv/source/*` 漏掉隐藏成员。执行后应检查：

```
find /srv/destination -mindepth 1 -maxdepth 2 -printf '%y %P -> %l\n' | sort
```

此命令只用于观察结果树；复杂筛选和批量处理的完整机制属于前一章。

④ [操作] 控制覆盖而不是事后补救

常见选项包括：

- `-i`：覆盖前询问；
- `-n`：不覆盖已有文件，但版本和脚本兼容性需要谨慎；
- `-u`：仅在源更新时复制，不能替代内容校验；
- `--backup`：覆盖前保留目标备份。

交互选项适合人工操作，不适合需要无人值守和确定终态的脚本。稳定流程是先列出将被覆盖的目标，再选择备份、跳过或明确替换策略。

⑤ [操作] 使用 `mv` 改名或移动

基本形式：

```
mv SOURCE DESTINATION
mv SOURCE... DIRECTORY
```

在同一文件系统内，移动通常主要改变目录项关系，速度很快；跨文件系统移动不能直接重命名，工具通常需要复制目标后删除源。跨文件系统时故障边界更复杂：目标可能已部分出现，而源仍然存在。因此重要移动后要同时验证：

```
test ! -e SOURCE
stat -- DESTINATION
```

并根据任务需要检查内容和元数据。不要仅因目标出现就认为源已安全删除。

⑥ [操作] 处理符号链接时明确“链接对象”还是“链接指向内容”

`cp` 的递归和解引用选项会影响符号链接。归档模式通常保留链接本身；显式解引用会复制链接指向的内容。操作前应通过：

```
stat -c '%F %N' -- PATH
```

确认对象类型。若任务要求保留目录树结构，默认应避免无意展开符号链接，否则可能复制超出预期集合的内容。

⑦ [验证] 复制和移动后的分层验证

推荐最小链：

- 源与目标存在状态
 - 目标层级和类型
 - 普通文件字节
 - 模式、所有者、组、时间戳
 - 链接关系

示例：

```
find /srv/destination -printf '%y %P -> %l\n' | sort
stat -c '%A %a %U %G %y %n' /srv/destination/important.conf
sha256sum /srv/source/important.conf /srv/destination/important.conf
```

[Cheatsheet] `cp` 保留源，`mv` 改变源路径；复制目录内容用 `SOURCE/. ; cp -a` 请求归档式保留但仍要验证；跨文件系统 `mv` 要检查两端；符号链接先确认复制链接还是目标内容。

[操作专题] 使用 `install` 把文件部署到明确终态

`install` 不是软件包管理器。它面向“把一个已准备好的文件部署到目标位置，同时明确创建父目录和设置目标属性”的任务。与先 `cp` 再 `chmod` 相比，它把内容复制和目标模式表达在同一操作中，更适合安装脚本、考试任务和可重复部署。

① [操作] 部署文件并指定模式

```
install -m 0640 app.conf /etc/myapp/app.conf
```

`-m` 设置目标模式，不是从源文件继承模式。若目标父目录已经存在，这条命令会复制内容并把目标模式设为 `0640`。

② [操作] 使用 `-D` 创建缺失的父目录

```
install -D -m 0640 app.conf /etc/myapp/app.conf
```

`-D` 为目标文件创建缺失的父目录。它适合“单个文件 + 完整目标路径”的部署。若需要独立创建目录并指定目录模式，应使用：

```
install -d -m 0750 /etc/myapp
```

③ [操作] 指定目标所有者和组

具有足够权限时可使用：

```
install -D -o root -g appops -m 0640 app.conf /etc/myapp/app.conf
```

目标用户和组必须存在；账号和组的创建属于下一章。普通用户无法任意设置所有者，因此参数存在不代表操作一定能完成。

④ [操作] 区分内容更新与目标属性语义

`install` 默认面向部署终态，不是通用的“原样克隆”工具。题目应先明确目标文件需要的模式、所有者和组，再通过参数显式设置；不要把源文件的偶然元数据直接当作部署终态。

⑤ [验证] 部署后同时验证内容与属性

```
sha256sum app.conf /etc/myapp/app.conf  
stat -c '%U %G %a %y %n' /etc/myapp/app.conf
```

摘要相同证明字节一致；`stat` 才回答所有者、组、模式和时间。若部署目录是服务配置目录，服务是否真正读取配置属于对应服务章节，不由文件存在自动证明。

[Cheatsheet] `install -m` 设置目标模式；`-D` 创建文件父目录；`-d` 创建目录；`-o/-g` 需要相应权限；`install` 面向部署终态，不等于完整保留源元数据。

[知识专题] `tar` 的成员、路径和元数据模型

`tar` 的核心对象不是压缩后缀，而是归档成员。创建归档时，源路径被转换为成员名称；查看归档时，成员名称决定未来提取出的层级；提取时，当前目录、目标目录、执行身份和保留选项共同决定恢复结果。最可靠的学习路径是先不压缩，理解创建、查看和提取，再叠加 `gzip`、`bzip2` 或 `xz`。

① [知识点] 归档是成员序列，压缩是归档流的表示方式

归档可以包含多个文件、目录、链接和元数据。压缩只把这个归档流变小。因而：

多个成员 → tar 归档 → 可选压缩 → 单个归档文件

单独执行 `gzip file` 会生成一个压缩文件，但不能把多个目录成员组织成可逐项列出和恢复的归档。

② [知识点] `c`、`t`、`x` 是互斥的主要操作

- `-c` / `--create`：创建；
- `-t` / `--list`：列出成员；
- `-x` / `--extract`：提取；
- `-f ARCHIVE`：指定归档文件。

命令的核心骨架是：

```
tar -cf archive.tar MEMBERS...
tar -tf archive.tar
tar -xf archive.tar
```

`-f` 后紧跟归档文件名。把归档名误当成员，或把成员误当归档名，是高频语法错误。

③ [知识点] 成员名称决定提取后的目录层级

若在 `/srv` 下执行：

```
tar -cf app.tar app
```

归档成员通常以 `app/` 开头。若进入 `/srv/app` 后归档 `.`，成员将以 `./` 开头，提取后不会自动形成名为 `app` 的顶层目录。任务要求固定顶层时，应从其父目录创建归档。

④ [知识点] `-C` 改变后续路径的解释位置

```
tar -C /srv -cf app.tar app
```

`-C /srv` 先改变 `tar` 处理后续成员时的工作目录，归档中保存 `app/...`，而不是本机绝对路径。`-C` 具有位置语义：若命令中多次出现，它只影响之后的路径参数，不能把它当成无顺序的装饰选项。

⑤ [知识点] 绝对路径成员具有覆盖风险

课程示例指出，`tar` 默认会从绝对源路径移除前导 `/`，以相对成员名称存储。这能降低提取时直接覆盖系统绝对路径的风险。即便工具有保护，未知归档仍应先列出成员并提取到空目录；不要把安全完全寄托在默认保护上。

⑥ [知识点] 归档元数据有默认范围和扩展范围

普通 tar 归档会保存常用模式、时间、所有者信息和链接类型，但 ACL、扩展属性和 SELinux 上下文通常需要显式选项，例如：

```
--acls
--xattrs
--selinux
```

保存了元数据也不代表任何用户都能原样恢复。root 与普通用户提取时的所有者结果不同；umask 和保留权限选项也会影响模式。

⑦ [知识点] 查看成员是创建和提取之间的安全闸门

```
tar -tf archive.tar.xz
tar -tvf archive.tar.xz
```

现代 tar 通常可从归档头识别压缩类型，因此只查看成员时常不必重复压缩选项。-t 不修改目标目录，适合先确认顶层、绝对路径、..、链接和成员集合。

[Cheatsheet] c/t/x 先选主要操作，f 后接归档名；成员名称决定提取层级；固定顶层从父目录归档或使用 -C；未知归档先 tar -tf；ACL、xattr 和 SELinux 元数据不是默认“全部保留”。

[操作专题] 创建、压缩、查看并安全提取归档

一个可靠的归档流程不是“执行一条 tar 命令”，而是先建立源集合基线，创建归档，非破坏性列出成员，在受控空目录试提取，再比较字节、层级和关键元数据。压缩算法的选择位于这个流程中间，不应遮蔽成员语义。

① [操作] 创建未压缩归档

```
tar -C /srv -cf /root/app.tar app
```

这条命令把 /srv/app 作为 app/... 成员写入 /root/app.tar。创建前确认目标归档文件是否已存在，因为同名归档可能被替换。

② [操作] 使用 gzip、bzip2 或 xz 压缩

对应短选项：

```
-z gzip    常见后缀 .tar.gz
-j bzip2   常见后缀 .tar.bz2
-J xz      常见后缀 .tar.xz
```

示例：

```
tar -C /srv -czf /root/app.tar.gz app
tar -C /srv -cjf /root/app.tar.bz2 app
tar -C /srv -cJf /root/app.tar.xz app
```

选项大小写必须精确，尤其是 `-j` 与 `-J`。

③ [操作] 独立压缩和解压单个文件

```
gzip report.txt
bzip2 report.txt
xz report.txt
```

通常会生成对应压缩文件并移除原始未压缩文件；需要保留原文件时应查看相应工具的保留选项，不要假设源一定仍在。解压可用：

```
gunzip report.txt.gz
bunzip2 report.txt.bz2
unxz report.txt.xz
```

`gzip -l` 和 `xz -l` 可在解压前查看压缩和未压缩大小，辅助评估空间，但不能证明内部 tar 成员正确。

④ [验证] 创建后列出成员和验证格式

```
file /root/app.tar.xz
tar -tf /root/app.tar.xz
tar -tvf /root/app.tar.xz
```

`file` 识别外层格式；`tar -tf` 验证成员名称；`tar -tvf` 增加类型、模式、所有者和时间等信息。三者证明的层次不同。

⑤ [操作] 提取到新建空目录

```
verify_dir=$(mktemp -d)
tar -C "$verify_dir" -xf /root/app.tar.xz
```

对压缩归档，tar 通常能自动识别算法。未知归档先在空目录提取，避免覆盖已有业务路径。提取后检查：

```
find "$verify_dir" -printf '%y %P -> %l\n' | sort
```


⑥ [操作] 控制权限和所有者恢复

普通用户提取时通常成为提取文件的所有者；root 可以恢复归档记录的所有者。当前 umask 可能收紧提取模式，保留权限选项可以改变该行为。任务要求精确模式时，必须在目标端用 `stat` 验证，不能只看 `tar` 没报错。

⑦ [验证] 比较源和试提取结果

先固定比较根，再生成普通文件摘要清单：

```
(
  cd /srv/app
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/source.sha256

(
  cd "$verify_dir/app"
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/extracted.sha256

cmp -s /tmp/source.sha256 /tmp/extracted.sha256
```

这只比较普通文件字节和相对名称。目录模式、链接目标、所有者和时间仍需独立检查。

[Cheatsheet] `-z/-j/-J` 分别对应 `gzip/bzip2/xz`；先 `file` 看外层，再 `tar -tf` 看成员；未知归档提取到空目录；root 与普通用户的所有者恢复不同；摘要清单不替代链接和元数据检查。

[操作专题] 使用 `scp` 与 `sftp` 理解远端路径

本章只讨论已具备 SSH 连通性和认证条件时的文件传输。SSH 服务端配置、密钥认证和主机密钥处理属于第 20 章。这里最重要的是分清本地端、远端端、传输方向和远端路径解释位置。

① [操作] 识别远端操作数

常见远端语法为：

```
[user@]host:path
```

上传：

```
scp local.tar.xz student@serverb:/home/student/incoming/
```

下载：

```
scp student@serverb:/home/student/incoming/result.txt ./
```

冒号左侧标识远端主机和可选用户，右侧是远端路径。远端路径为空或相对时可能落入远端用户主目录语境，不应凭本地路径推测。

② [操作] 使用绝对路径消除远端落点歧义

考试和变更任务中，优先使用明确绝对路径：

```
scp archive.tar.xz student@serverb:/home/student/incoming/archive.tar.xz
```

执行前可先通过 SSH 验证目标：

```
ssh student@serverb 'pwd; stat -c "%F %a %U %G %n" /home/student/incoming'
```

此处只把 SSH 当作远端证据入口，不展开 SSH 配置。

③ [操作] 递归传输目录和保留时间戳

`scp -r` 递归复制目录；`scp -p` 请求保留修改时间、访问时间和模式。不同权限和后端下的属性保留仍需远端 `stat` 验证。对复杂目录树、增量同步和删除策略，应优先考虑 `rsync`，而不是反复完整复制。

④ [操作] 使用交互式 `sftp`

```
sftp student@serverb
```

会话中本地和远端各有独立工作目录：

<code>pwd / cd</code>	远端工作目录
<code>lpwd / lcd</code>	本地工作目录
<code>put</code>	上传
<code>get</code>	下载
<code>mkdir</code>	创建远端目录
<code>ls / lls</code>	查看远端 / 本地

开始传输前同时执行 `pwd` 和 `lpwd`，能显著减少方向和落点错误。

⑤ [操作] 递归上传或下载目录

交互式示例：

```
sftp> lcd /srv/export
sftp> cd /home/student/incoming
sftp> put -r app
```

或下载：

```
sftp> lcd /var/tmp/review
sftp> get -r /home/student/result
```

传输进度为 100% 只能说明客户端完成了传输流程，仍需在目标端检查层级、字节和必要属性。

⑥ [知识点] RHEL 9 中 `scp` 命令与底层协议要分开

RHEL 9 的 OpenSSH `scp` 默认使用 SFTP 协议实现传输；旧 SCP 协议可通过兼容选项强制使用。课程扫描材料中的安全警告反映了旧 SCP 协议的历史问题，但不能简单推出“RHEL 9 的 `scp` 命令一定使用旧协议”。本章保留 `scp` 命令的考试与运维语义，同时把底层协议差异标记为版本敏感点。

⑦ [验证] 远端传输后的最小验收

```
ssh student@serverb '
stat -c "%F %a %U %G %s %y %n" /home/student/incoming/archive.tar.xz
sha256sum /home/student/incoming/archive.tar.xz
'
```

本地计算同一文件摘要后比较。若目标是目录，还要检查是否多包一层，以及隐藏成员是否存在。

[Cheatsheet] 远端语法是 `[user@]host:path`；优先使用绝对路径；`sftp` 有本地和远端两套工作目录；100% 进度不等于终态正确；RHEL 9 的 `scp` 默认后端是版本敏感事实，和命令界面分开理解。

[操作专题] 使用 `rsync` 同步差异并控制删除边界

`rsync` 的核心不是“更快的 `scp`”，而是比较源集合与目标集合，根据选择规则和属性策略决定哪些对象需要创建、更新、跳过或删除。首次同步可能接近完整复制，后续同步才体现差异传输优势。任何涉及目标覆盖或删除的命令，都应先 `dry-run`。

① [知识点] 源目录末尾斜杠定义同步根

```
rsync -a /srv/source /srv/target/
```

结果通常包含：

```
/srv/target/source/...
```

而：

```
rsync -a /srv/source/ /srv/target/
```

表示把 `source` 的内容直接同步到目标：

```
/srv/target/...
```

这不是格式偏好，而是结果层级语义。执行前应先写出预期目录树。

② [操作] 使用归档模式同步常用属性

```
rsync -a SOURCE/ DESTINATION/
```

`-a` 等价于一组常用选项，通常包含递归、保留符号链接、模式、时间、组、所有者和设备等意图。但它不包含所有扩展属性：

- `-H`：保留硬链接关系；
- `-A`：保留 ACL；
- `-X`：保留扩展属性，SELinux 上下文通常也依赖 `xattr`；

这些选项会增加成本和权限要求，只在任务需要时使用。

③ [查询] 先用 `dry-run` 和逐项输出审查计划

```
rsync -ani /srv/source/ student@serverb:/home/student/stage/
```

- `-n` / `--dry-run`：不执行更改；
- `-i` / `--itemize-changes`：逐项说明将发生的变化；
- `-v`：增加过程信息。

检查重点：目标主机和路径、源末尾斜杠、将创建的顶层、将覆盖的对象以及是否出现删除项。

④ [操作] 执行本地或远端同步

本地：

```
rsync -a /srv/source/ /srv/target/
```

上传到远端：

```
rsync -a /srv/source/ student@serverb:/home/student/stage/
```

从远端下载：

```
rsync -a student@serverb:/home/student/result/ /var/tmp/result/
```

远端两端不能都同时是远端操作数；至少一端是本地执行环境。

⑤ [边界] `--delete` 是镜像策略，不是默认同步选项

`--delete` 允许删除目标端在源集合中不存在的成员。它只适用于目标明确要求“精确镜像源”的场景。风险组合包括：

- 源路径写错或为空；
- 目标路径写错；
- 源末尾斜杠改变同步根；
- 排除规则改变可见源集合；
- 在未 `dry-run` 时直接执行。

安全流程：

```
rsync -ani --delete SOURCE/ DESTINATION/  
# 审查所有 *deleting 项后，再决定是否执行
```

若任务要求保留远端额外文件，必须明确排除 `--delete`。

⑥ [验证] 第二次 `dry-run` 无变化只证明 `rsync` 的比较规则已收敛

执行后再次运行相同 `dry-run`：

```
rsync -ani SOURCE/ DESTINATION/
```

无输出通常说明 `rsync` 根据当前比较策略认为无需更新。默认快速检查主要依赖大小和修改时间等元数据；它不是独立的全量字节证明。高价值文件仍可使用 `sha256sum` 抽查或建立清单。

⑦ [知识点] 所有者保留需要目标端权限

`-a` 包含所有者和组保留意图，但普通远端用户通常不能创建任意 `UID/GID` 所有者。若题目只要求内容、模式和时间，不应把源 `root` 所有者当作必须终态；若确实要求所有者，需确保目标端以足够权限运行，并在目标端验证。

⑧ [诊断] 反复同步同一文件时先比较大小、时间和两端时钟

若每次 `dry-run` 都显示同一文件变化：

1. 用 `stat` 比较大小和纳秒级修改时间；
2. 检查文件是否被应用持续改写；
3. 检查两端时钟和文件系统时间精度；
4. 再决定是否内容校验或调整比较策略。

不要一开始就用强制校验所有文件掩盖持续写入或时间异常。

[Cheatsheet] 源尾随斜杠决定同步根；`-a` 不包含硬链接、ACL 和全部 xattr；先 `-n -i` 再执行；`--delete` 只用于明确镜像目标；第二次 dry-run 是差异收敛证据，不等于全部字节和属性已经独立证明。

[操作专题] 使用 `sha256sum` 建立字节证据并完成分层验收

摘要是非常有价值的内容证据，但它的边界必须清楚。摘要函数把一个字节序列映射为固定长度值；比较同一算法的摘要，可以高置信度判断字节是否相同，却不会自动描述路径、权限或来源真实性。

① [操作] 计算单个文件摘要

```
sha256sum archive.tar.xz
```

输出包括摘要和文件名。传输前后应对同一个逻辑文件分别计算，不要把不同压缩算法生成的归档摘要直接要求相同。

② [操作] 创建和检查摘要清单

```
sha256sum file1 file2 > SHA256SUMS
sha256sum -c SHA256SUMS
```

检查清单时，文件名按清单记录的路径解释。移动清单或改变当前目录后，应确认路径仍能解析到目标文件。

③ [边界] 摘要清单本身必须可信

`sha256sum -c` 只说明当前文件与给定清单一致。若清单和文件由同一不可信来源同时被修改，检查仍可能通过。数字签名、可信发布通道和供应链真实性不在本章完整展开，但不能把普通摘要称为“身份认证”。

④ [操作] 对目录建立稳定的普通文件摘要视图

目录不是一个单一字节流。可先固定比较根和成员顺序：

```
(
  cd /srv/source
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/source.sha256
```

在目标根执行同样流程，再使用 `cmp` 或 `diff` 比较清单。该方法把相对名称纳入文本清单，但仍不检查目录、符号链接和元数据。

⑤ [验证] 组合成员、字节和元数据证据

完整验收示例：

```
find TARGET -printf '%y %P -> %l\n' | sort
sha256sum SOURCE_FILE TARGET_FILE
stat -c '%F %a %U %G %y %n' TARGET_FILE
```

- `find`：层级、类型和链接目标；
- `sha256sum`：普通文件字节；
- `stat`：模式、所有者、组和时间。

⑥ [知识点] 归档摘要与归档成员摘要回答不同问题

归档文件摘要相同，证明整个归档字节相同；两个归档成员内容相同，却可能因为成员顺序、时间戳或压缩参数不同而得到不同的归档摘要。任务若要求“传输文件未改变”，比较归档摘要足够；若要求“恢复后的数据集正确”，还需试提取和成员级验证。

[Cheatsheet] `sha256sum` 只看字节；`-c` 相对清单检查；清单本身不提供来源真实性；目录先固定成员集合和相对根；归档摘要和恢复后成员验证回答不同问题。

[诊断专题] 从“文件不对”推进到最有区分度的证据

“复制失败”“归档不对”或“同步异常”都过于宽泛。诊断应先把症状归入路径层级、成员集合、字节、元数据、远端落点或删除范围，再选择下一条最有区分度的证据。不要反复重跑命令，因为重跑可能继续覆盖或删除目标证据。

① [诊断] 目标多出一层目录

症状：预期 `/target/a.conf`，实际为 `/target/source/a.conf`。

当前证据：

```
find /target -mindepth 1 -maxdepth 2 -printf '%y %P\n' | sort
```

假设：`rsync` 源缺少尾随斜杠，或复制目录到已有目标目录时把目录本身作为成员。

最小修复：先重新写出期望树，调整源根；不要直接把所有文件移动一层，除非已确认目标中没有其他合法成员。

② [诊断] 隐藏成员缺失

症状：普通文件存在，但 `.env` 或 `.config` 不在目标。

当前证据：

```
find SOURCE -mindepth 1 -maxdepth 1 -printf '%P\n' | sort
find TARGET -mindepth 1 -maxdepth 1 -printf '%P\n' | sort
```

假设： 使用了 `SOURCE/*`，Shell 展开漏掉点号成员。

最小修复： 使用 `SOURCE/.` 或直接把目录交给能够递归处理的工具。

③ [诊断] 归档可读取但成员顶层错误

症状： `tar -tf` 成功，但成员以 `./` 开头或缺少要求的顶层目录。

下一条证据： 只查看成员前若干行和创建命令所处目录。

```
tar -tf archive.tar.xz | sed -n '1,30p'
pwd
```

假设： 在源目录内部归档 `.`，或 `-C` 和成员参数组合错误。

最小修复： 从源目录父级重建归档，例如 `tar -C /srv -cJf ... app`。

④ [诊断] 摘要相同但权限或时间错误

症状： 内容检查通过，应用仍拒绝读取或任务要求的时间未保留。

下一条证据：

```
stat -c '%F %a %U %G %y %n' SOURCE TARGET
```

假设： 未请求保留、执行身份不足、目标文件系统不支持，或提取时受到 `umask` 影响。

最小修复： 只修正题目要求的属性，并再次验证；权限模型的完整讲解归后续权限章节。

⑤ [诊断] 传输显示成功但远端找不到文件

症状： 客户端显示 100%，预期绝对路径不存在。

下一条证据：

```
ssh USER@HOST 'id; pwd; find "$HOME" -maxdepth 3 -name "EXPECTED" -print'
```

假设： 使用了远端相对路径、登录用户与预期不同，或在 SFTP 会话中远端工作目录已改变。

最小修复： 使用明确绝对路径，或在 `sftp` 中先 `pwd` 后 `put/get`。

⑥ [诊断] `rsync --delete` 计划删除大量对象

症状： `dry-run` 出现大量 `*deleting`。

下一条证据：


```
printf 'SOURCE=%q\nDEST=%q\n' "$src" "$dst"
find "$src" -mindepth 1 -maxdepth 2 -printf '%P\n' | sort | sed -n '1,50p'
```

同时确认远端路径和源末尾斜杠。

最小修复：停止实际执行，修正同步根或目标；不得用无调查的强制选项继续。

⑦ [诊断] `rsync` 仍反复显示变化

当前证据：`rsync -ani` 的逐项变化类型。

下一条证据：比较两端大小、修改时间、文件是否持续写入以及时钟状态。

最小修复：先消除持续写入或时间差，再决定是否使用内容校验。不要用更激进的同步掩盖上游状态变化。

⑧ [诊断] `tar` 报压缩格式不匹配

症状：使用 `-z` 提取 `.tar.xz` 时出现“not in gzip format”。

下一条证据：

```
file archive
tar -tf archive
```

假设：手工指定的算法与实际文件不匹配，或扩展名错误。

最小修复：让 `tar` 自动识别，或使用与实际格式对应的参数；不要仅根据文件名后缀下结论。

⑨ [诊断] `install` 内容正确但服务仍不工作

当前证据：摘要和 `stat` 已满足文件终态。

判断边界：本章到此只能证明部署文件正确。应用是否读取该路径、配置语法是否有效、服务是否重载属于对应服务和应用章节。下一条证据应转向应用自己的验证命令或日志，而不是继续重复复制。

[Cheatsheet] 先把症状归类为层级、成员、字节、元数据、远端落点或删除范围；下一条证据应能区分假设；重跑可能破坏证据；最小修复后按原验收层重新验证。

[经典任务] 创建保留关键属性的 `xz` 压缩归档并完成试提取验收

环境与当前状态

服务器上已有发布目录：

```
/srv/app-release/  
├─ .release-env  
├─ bin/start.sh  
├─ conf/app.conf  
├─ data/banner.txt  
└─ current -> data
```

现在需要把它交付给另一名管理员。`/root/submission/` 已存在，里面可能有旧文件，但不存在本次日期命名的目标归档。以下流程先明确预期成员和验收证据，再给出推荐操作；只有在目标主机实际执行并收集证据后，才能声明任务完成。

目标终态

- 1. 在 `/root/submission/` 创建 `app-release-YYYY-MM-DD.tar.xz`；
- 2. 归档使用 `xz` 压缩；
- 3. 所有成员必须位于单一顶层 `app-release/` 下；
- 4. 必须包含隐藏文件 `.release-env`；
- 5. 不能保存以 `/srv/` 开头的绝对成员名；
- 6. 不得修改源目录；
- 7. 创建后先列出成员，再提取到新建空目录；
- 8. 验证普通文件字节、符号链接目标和 `bin/start.sh` 模式；
- 9. 不能把“归档文件存在”作为唯一验收。

限制条件

- 不直接提取到 `/srv`、`/` 或已有业务目录；
- 不使用 `*` 作为成员集合；
- 不关闭 SELinux、不使用 `chmod 777`；
- 不把示例命令或单条成功信息当作已经完成的目标端验收。

验收证据

层次	推荐证据
归档对象	<code>stat</code> 、 <code>file</code>
成员与顶层	<code>tar -tf</code>
试提取层级	<code>find</code>
普通文件字节	稳定相对摘要清单
链接	<code>find -type l -printf</code> 或 <code>readlink</code>
关键模式	<code>stat -c '%a %n'</code>

[参考解答] 经典任务一

① 调查并建立源基线

```
src=/srv/app-release
out=/root/submission
stamp=$(date +%F)
archive="$out/app-release-$stamp.tar.xz"

stat -- "$src" "$out"
test ! -e "$archive"
find "$src" -printf '%y %P -> %l\n' | sort
stat -c '%F %a %U %G %y %n' "$src/bin/start.sh"
```

`test ! -e` 防止无意覆盖同名交付物。`find` 同时显示隐藏成员和符号链接，避免用 `ls` 或 `*` 形成不完整基线。

② 从父目录创建具有固定顶层的归档

```
tar -C /srv -cJf "$archive" app-release
```

参数解释：

- `-C /srv`：把后续成员从 `/srv` 解释；
- `-c`：创建归档；
- `-J`：使用 `xz`；
- `-f "$archive"`：指定归档文件；
- `app-release`：归档成员以该顶层开始。

③ 非破坏性验证外层和成员

```
stat -- "$archive"
file -- "$archive"
tar -tf "$archive"
```

应确认：首层为 `app-release/`，列表中有 `app-release/.release-env`，不存在以 `/srv/` 或 `/` 开头的成员。

可做静态成员检查：

```
members=$(mktemp)
tar -tf "$archive" > "$members"
grep -Fx 'app-release/.release-env' "$members"
! grep -E '^|(^|/)\.\\. (|/)$' "$members"
```

④ 提取到新建空目录

```
verify_dir=$(mktemp -d)
tar -C "$verify_dir" -xf "$archive"
find "$verify_dir" -printf '%y %P -> %l\n' | sort
```

此时预期根为 `$verify_dir/app-release`。

⑤ 比较普通文件字节

```
(
  cd /srv/app-release
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/app-source.sha256

(
  cd "$verify_dir/app-release"
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/app-extracted.sha256

diff -u /tmp/app-source.sha256 /tmp/app-extracted.sha256
```

没有差异只证明相对名称对应的普通文件字节一致。

⑥ 验证链接和关键模式

```
readlink /srv/app-release/current
readlink "$verify_dir/app-release/current"
stat -c '%a %n' /srv/app-release/bin/start.sh \
  "$verify_dir/app-release/bin/start.sh"
```

若任务还要求 ACL、扩展属性或 SELinux 上下文，需要在创建和提取时加入对应 `tar` 选项，并使用 `getfacl`、`getfattr` 或 `ls -Z` 单独验收。本任务只要求关键普通模式和链接目标，因此不扩大范围。

⑦ 结果声明边界

只有在实际执行并取得成员、路径、字节和元数据证据后，才能声明任务完成。示例流程本身不构成运行证据；在没有目标主机输出时，应把结论限定为“已给出推荐操作与验收方法”。

[经典任务] 安全同步目录到远端并证明源目录末尾斜杠语义正确

环境与当前状态

本地源目录：

```
/srv/site/  
├── .well-known/status  
├── index.html  
└── assets/logo.txt
```

远端目标目录：

```
student@serverb:/home/student/stage/site/
```

目标目录已存在，并有一个远端管理员留下的 `remote-note.txt`。本次任务要求更新源中已有内容，但不得删除该远端额外文件。

目标终态

1. `/srv/site/` 的成员直接进入远端 `site/`；
2. 不能形成 `site/site/`；
3. 隐藏目录 `.well-known` 必须同步；
4. 保留普通模式和修改时间；
5. `remote-note.txt` 必须保留；
6. 实际执行前必须 `dry-run`；
7. 执行后再次 `dry-run`，不应出现预期外差异；
8. 抽查 `index.html` 的 SHA-256；
9. 不要求把本地 `root` 所有者映射为远端 `root`。

限制条件

- 不使用 `--delete`；
- 不修改 SSH 服务端配置；
- 不把一次 `rsync` 成功扩大成远端业务可用；
- 不编造服务器地址、输出或实时验证结果。

[参考解答] 经典任务二

① 调查两端和写出预期层级

本地：

```
find /srv/site -printf '%y %P -> %l\n' | sort
```

远端：

```
ssh student@serverb '
  id
  stat -c "%F %a %U %G %n" /home/student/stage/site
  find /home/student/stage/site -mindepth 1 -maxdepth 2 -printf "%y %P -> %l\n" | sort
'
```

预期是 `index.html`、`assets/` 和 `.well-known/` 直接位于远端 `site/`，因此源必须写成 `/srv/site/`，带末尾斜杠。

② 先 dry-run 并逐项审查

```
rsync -ani /srv/site/ \
  student@serverb:/home/student/stage/site/
```

检查清单中不应出现：

- 目标形成新的 `site/` 顶层；
- `*deleting remote-note.txt`；
- 意外远端路径。

本任务禁止 `--delete`，因此远端额外文件应保留。

③ 执行同步

```
rsync -a /srv/site/ \
  student@serverb:/home/student/stage/site/
```

`-a` 请求递归并保留常用模式、时间和链接等属性。普通远端用户不能任意恢复本地 `root` 所有者，因此本任务不把所有者映射列为验收目标。

④ 再次 dry-run 验证差异收敛

```
rsync -ani /srv/site/ \
  student@serverb:/home/student/stage/site/
```

无预期外输出说明 `rsync` 的当前比较规则认为两端不需再次更新；这不替代字节抽查。

⑤ 验证远端层级和额外文件

```
ssh student@serverb '
  find /home/student/stage/site -mindepth 1 -maxdepth 2 -printf "%y %P -> %l\n" | sort
  test -f /home/student/stage/site/remote-note.txt
  test ! -e /home/student/stage/site/site
'
```

⑥ 抽查字节、模式和时间

本地：

```
sha256sum /srv/site/index.html
stat -c '%a %y %n' /srv/site/index.html
```

远端：

```
ssh student@serverb '
sha256sum /home/student/stage/site/index.html
stat -c "%a %y %n" /home/student/stage/site/index.html
'
```

比较摘要、模式和修改时间。若摘要相同而时间不同，应回到 `rsync` 输出和目标文件系统能力调查，而不是直接宣布任务失败或强制覆盖。

⑦ 镜像变式的边界

只有题目明确要求“远端必须与源完全一致，不得保留任何额外成员”时，才考虑：

```
rsync -ani --delete /srv/site/ student@serverb:/home/student/stage/site/
```

必须先人工审查所有删除项，再执行实际同步。本题存在必须保留的 `remote-note.txt`，所以 `--delete` 是错误答案。

[本章收束] 从命令记忆转向数据集合的证据闭环

本章的共同对象不是某个命令，而是从源集合到目标集合的一次可验证变换。无论使用 `cp`、`tar`、`scp` 还是 `rsync`，都应先明确源边界和目标层级，再决定是否保留元数据、是否允许覆盖或删除，最后用互不替代的证据关闭任务。

确认源集合和目标边界

- 选择复制、部署、归档、传输或同步模型
- 控制层级、覆盖、属性和删除策略
- 先预览或建立基线
- 执行最小操作
- 验证成员、路径、字节和元数据

工作方法：把模糊的“搬过去”改写成四个可判定问题

1. 搬什么？是单个对象、目录本身、目录内容，还是归档成员集合？
2. 搬到哪里？目标路径由本地还是远端解释，目标是否已存在，顶层应长什么样？
3. 保留什么？只要求字节，还是还要求模式、属主、时间戳、链接、ACL 或扩展属性？

4. 怎样证明？用成员表证明集合，用路径树证明层级，用摘要证明字节，用 `stat` 证明元数据。

主要判断表

目标或现象	首选入口	关键判断	最少再验证
本地创建副本	<code>cp</code>	目录本身还是目录内容	层级、摘要、 <code>stat</code>
本地改名或移动	<code>mv</code>	是否跨文件系统	两端存在状态、目标内容
部署文件并指定属性	<code>install</code>	目标目录和模式是否明确	摘要、模式、属主
把多个成员封装为单文件	<code>tar</code>	成员前缀和顶层是否正确	<code>tar -tf</code> 、试提取
压缩一个文件或归档流	<code>gzip</code> / <code>bzip2</code> / <code>xz</code>	压缩不是归档	<code>file</code> 、可解压性
一次性远端复制	<code>scp</code>	远端路由由谁解释	远端 <code>stat</code> 、层级
交互式远端浏览和传输	<code>sftp</code>	本地/远端工作目录不同	<code>lpwd</code> 、 <code>pwd</code> 、目标检查
目录差异同步	<code>rsync</code>	源末尾斜杠和删除范围	dry-run、第二次比较、摘要
证明普通文件字节	<code>sha256sum</code>	摘要不证明元数据	<code>stat</code> 、成员和路径检查

向下一章交接

本章已经能够把数据集送到正确位置并证明内容、层级和必要属性。下一章《用户、组与账号生命周期》将回答另一个问题：这些文件由谁拥有、哪些账号应存在、用户和组的生命周期如何改变访问边界。本章只在部署和传输时引用现有用户身份，不提前展开账号创建、密码策略和组成员关系。